

TP: Introduction à R

1 Une introduction au logiciel R

1.1 Séquences, Vecteurs, Matrice Tableaux (arrays)

Pour obtenir l'aide de R sur une commande, il faut taper `help("nom de la commande")`. L'aide est en anglais.

Pour rentrer un vecteur, on peut utiliser les commandes `c`, `seq` ou `rep`. Pour rentrer une matrice, on utilise la commande `matrix(nomvecteur, nombre lignes, nombre colonnes, sens de remplissage)`.

Le logiciel R, comme le logiciels Python, est capable de faire des opérations sur les vecteurs et les matrices. Il faut cependant faire très attention :

La commande `A*B` n'est pas une multiplication matricielle, on se contente de multiplier les coefficients entre eux. De même, quand on tape `A^2`, on ne calcule pas le carré de la matrice A, on met juste les coefficients de la matrice A au carré.

Pour faire une vraie multiplication matricielle, il faut taper `A%*%B`. La commande `solve` permet d'inverser une matrice.

La fonction `outer(u,v,'f')` permet à partir de deux vecteur u de longueur m et v de longueur n de créer la matrice $f(u_i, v_j)_{ij}$.

Exercice 1 : Déclaration et définition

1. Saisir le vecteur a en tapant `a <- c(10, 5, 3, 6, 21)`. Taper a . Puis $a[2]$, $a[1,3]$ et $a[c(1,3)]$.
2. Créer un vecteur b grâce à la commande `b<-matrix(c(1,1,1,1,1),ncol=1)`.
3. Générer un vecteur c de dimension 5 dont toutes les composantes sont 1 grâce à la commande `rep`.
4. Générer un vecteur d en tapant `d<-seq(from=1, to=10, by=2)`.
5. Si on veut générer des séquences d'entiers consécutifs, on peut utiliser la commande `1:5`. Créer un vecteur e de dimension 5 et de composantes la séquence des 5 premiers entiers.
6. Générer une matrice diagonale de dimension 3 dont les éléments de la diagonale sont 1, 5 et 9 à l'aide de la commande `diag`. Demander de l'aide sur la fonction `diag` grâce à la commande `help`.
7. Créer la matrice

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 3 & 1 \end{pmatrix}$$

grâce à la commande `matrix`.

Exercice 2 : Manipulations de base

1. Taper `2*a+b+1`. Qu'obtient-on ?
 2. Taper `e[3]`. Qu'obtient-on ?
 3. Taper `cos(a)`, `exp(a)`. Qu'obtient-on ?
 4. Taper `a*e` puis `a**e`. Comparer.
 5. Taper maintenant `b**e`. Que se passe-t-il ? Corriger par `f<- t(b)**e`. Que fait la commande `t` ?
 6. Demander `dim(f)`. Taper `f[2,3]`, `f[,3]`, `f[2:5,]` puis `f[2:3,4]`. Qu'obtient-on ?
 7. Taper `cbind(b,e)` puis `rbind(b,e)`. Pourquoi faut-il éviter d'utiliser `cbind` ou `rbind` avec des objets du type de `a` ? (Essayer `cbind(a,b)` et `rbind(a,b)`).
 8. Taper `c(a,b)`. Qu'obtient-on ?
- Les listes permettent de stocker plusieurs éléments de type différent.

Exercice 3 : Listes

1. Taper `Lst <- list(name="Fred", wife="Mary", no.children=3, child.ages=c(9,7,4))`. Demander `Lst`.
2. Demander `Lst[[1]]`. Retrouver le résultat en tapant `Lst$name`
3. Demander `Lst[[4]]`. Qu'obtient-on. Demander l'âge du troisième enfant.

Exercice 4 : Résolution d'un système linéaire et inversion d'une matrice carrée

Soient :

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 3 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}.$$

1. Saisir la matrice A (par colonnes) et le vecteur b . La matrice A est-elle inversible ? Calculer son déterminant grâce à la commande `det(A)`.
2. Regarder l'aide de la commande `solve` et résoudre $Ax = b$.
3. Pour `eigen(A, symmetric=FALSE, only.values = FALSE)`. Le résultat de cette commande est une liste. En extraire les valeurs propres, dans une matrice que l'on appellera D et les vecteurs propres dans une matrice de passage que l'on appellera P .
4. calculer P^{-1} à l'aide de la commande `solve(P)` et retrouver A à partir de P , D et P^{-1} .

1.2 Boucles for, structures if then else et fonctions

Structure d'une boucle for `for (k in 1:n)`

```
{  
  lignes codes  
}
```

Les accolades ne sont pas nécessaires s'il n'y a qu'une seule ligne de code.

Conditions `if(j<=i+1)`

```
{  
  lignes codes  
}
```

```

else
{
lignes codes
}

```

Boucles while while (condition)

```

{
lignes commandes
}

```

Fonctions nomfonction <- function (nomvariables)

```

{
une ligne de commandes
}

```

ou si la fonction est plus compliquée :

```

nomfonction<- function (nomvariables)
{
lignes commandes
nomfonction<- expression
}

```

Dans ce dernier cas, le résultat de la fonction ne s'affichera pas automatiquement.

Exercice 5 : Fonctions statistiques

1. Créer une fonction **moyenne** qui calcule la moyenne d'un vecteur v . On pourra utiliser la fonction **sum**. La tester.
2. Créer une fonction **moyenne_pond** qui calcule une moyenne pondérée (les notes sont dans le vecteur v , les coefficients dans le vecteur c). La tester.
3. Créer une fonction **variance** qui calcule la variance d'un vecteur.
4. Créer une fonction **variance_pond** qui calcule une variance pondérée.
5. Vérifier ces fonctions sur le vecteur suivant : (10, 8, 12, 10) avec les pondérations (2, 2, 1, 1).

Exercice 6 : Jeu

On modifie un nombre comme suit :

Si n est pair, on le divise par 2, si n est impair, on le multiplie par 3 et on lui ajoute 1. On répète ensuite l'opération.

1. Appliquer ces opérations sur différents nombres : 4, 5, 8 par exemple (faire les calculs de tête ou à la main). Que peut-on remarquer ?
2. Créer une fonction **jeu** qui, étant donné un nombre n , affiche la suite de ses 10 premières transformations. On pourra utiliser la commande **n%%p** qui donne le reste de la division de n par p .
3. Créer une fonction **jeu2** qui, étant donné un nombre n , affiche la suite de ses transformations jusqu'à ce qu'on atteigne le nombre 1.
4. Créer une fonction **nombre** qui, étant donné un nombre n , donne le nombre d'opérations que l'on doit faire avant d'atteindre le nombre 1.
5. Chercher, parmi les cent premiers nombres entiers, celui pour lequel la fonction **nombre** atteint son maximum. Quel est ce maximum ? (On pourra utiliser les fonctions **max** et **which.max**).

1.3 Graphiques et bases de données

Le logiciel R permet d'importer des tableaux de données. Il existe pour cela plusieurs commandes : `read.table` est la plus générale, `read.csv` `read.csv2` ont des options déjà prédéfinies. Le format `.csv` est le format de base des tableaux (comme `.txt` pour les textes), il peut être produit par tous les tableurs comme Excel ou OpenOffice.

Le logiciel R permet aussi de faire beaucoup de graphiques différents :

- `pie` : camembert
- `hist` : histogramme
- `boxplot` : boîte à moustaches
- ...

Pour tracer un grapique classique, on utilise `plot` pour le premier tracé, puis `lines` ou `points` pour les suivants, qui se superposent. Pour effacer la figure, il suffit de relancer un `plot`.

Exercice 7 : Salaires

Télécharger le fichier `salaires.csv`. Le fichier vient de l'INSEE, et est encodé comme suit :

- CS : catégories socio-professionnelles (5 : cadres et professions intellectuelles supérieures, 4 : professions intermédiaires, 3 : employés, 6 : ouvriers qualifiés, 7 : ouvriers non qualifiés.)
- : CE : conditions d'emploi : (C : temps complet, N : temps non complet).
- : Age : c'est l'âge maximum de la catégorie considérée.
- salaire : salaire moyen brut équivalent temps plein pour chaque catégorie
- nombre : nombre d'emplois (équivalent temps plein) pour chaque catégorie.

1. Télécharger le dataframe "salaires". Utiliser la commande `sal<-read.csv("salaires.csv")`. Avant, il faut se placer dans le bon répertoire de travail.
2. Afficher la colonne des salaires. `sal$salaire`, `sal[,5]`, `attach(sal)`, `salaire`. Commenter.
3. Taper `boxplot(salaire)`, `boxplot(salaire~SEXE)`. Commenter.
4. Tracer les boîtes à moustaches du salaire en fonction de la catégorie socio-professionnelle. Comparer les différences entre les hommes et les femmes, puis en fonctions des catégories socio-professionnelles.
5. Taper `hist(Salaire)`.

Exercice 8 : Graphique

1. Créer un vecteur $x = (0, 0.01, 0.02 \dots, 2\pi)$ à l'aide de la commande `seq`.
2. Taper `plot(x,sin(x),type='l',col='blue',ylim=c(-1,1))`
3. Taper `lines(x,cos(x),col='red')`. Que se passe-t-il si on utilise la commande `plot` et non la commande `lines` ?
4. Refaire la même chose avec $x = (0, 0.1, 0.2 \dots, 2\pi)$ en utilisant `type=p` et la commande `points` au lieu de `lines`. Commenter.

1.4 Loïs de probabilités

R connaît la plupart des lois de probabilités usuelles. On y accède en chargeant le package `stats` (`library(stats)`). Il peut :

- Simuler une variable suivant cette loi grâce à la commande `rloi`

- Donner la densité : `dloi`
- Donner la fonction de répartition : `ploi`
- Donner la fonction quantile : `qloi`.

Exercice 9 : Calculs de probabilité

1. Simuler 1000 variables de loi normale. Utiliser les fonctions `boxplot` puis `hist` pour visualiser les résultats.
2. Donner la probabilité qu'une loi binomiale de paramètres $(10, 0.7)$ soit plus grande que 8 puis la probabilité qu'elle soit plus petite que 4.
3. Trouver z tel que $\mathbb{P}(E > z) \leq 0.05$ où E suit une loi exponentielle de paramètre 2.

Exercice 10 : Simulation de lois normales non indépendantes

On veut générer $n = 100$ variables aléatoires indépendantes de $\mathbb{R}^3$ de loi $\mathcal{N}(m, \Sigma)$ où

$$m = \begin{pmatrix} 5 & 3 & 10 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 3 & -0.5 & 3 \\ -0.5 & 1.25 & 0.5 \\ 3 & 0.5 & 5 \end{pmatrix}$$

On veut que les variables de $\mathbb{R}^3$ aient pour espérance m et pour matrice de variance/covariance Σ . La matrice de variance/covariance théorique d'un vecteur X peut être définie comme

$$\text{Cov}(X) = \mathbb{E} \left({}^t(X - \mathbb{E}(X))(X - \mathbb{E}(X)) \right)$$

Malheureusement, le logiciel **R** ne permet de générer que des variables aléatoires de loi normale indépendantes.

1. Rappeler comment simuler un vecteur de loi normal de matrice de variance/covariance quelconque. Calculez la racine carrée de Σ à l'aide de la fonction `eigen` de **R**. La commande `diag` permet de créer une matrice diagonale à partir d'un vecteur.
2. À l'aide de la fonction `rmnorm` et de S , simuler n variables i.i.d de $\mathbb{R}^3$ de loi $\mathcal{N}(m, \Sigma)$. On stockera ces variables dans une matrice de taille $n \times 3$. Ne pas faire de boucle pour cette question, il est possible de travailler uniquement avec des vecteurs!.
3. À l'aide des fonctions `colMeans` et `var`, vérifier que la moyenne empirique et la matrice de variance/covariance empirique du jeu de données Y est assez proche de la matrice A . Vous pouvez relancer le programme avec $n = 1000$ si vous trouvez les résultats trop imprécis.
4. Calculer la matrice de corrélation théorique avec le logiciel, et la comparer au résultat empirique obtenu.